

MACHINE LEARNING ENGINEERING

FASE 1 | AULA 06 -

IMPLANTAÇÃO DE APIS: CONTAINER E CLOUD

SUMÁRIO

O QUE VEM POR AÍ?	3
HANDS ON	4
SAIBA MAIS.....	5
O QUE VOCÊ VIU NESTA AULA?	31
REFERÊNCIAS.....	32

EMAN

O QUE VEM POR AÍ?

Agora que temos uma API robusta no ambiente local, vamos aprender a implantar nossa API com Docker e colocá-la em produção no Render!

Abram o VSCode, assistam os vídeos e se preparem, hoje vamos até o deploy!

EMAND

HANDS ON

Preparem-se para mais uma aula hands on, onde vamos melhorar nossa API FastAPI e explorar containerização da API (Docker), além do deploy em nuvem com um exemplo prático!

EMSE

SAIBA MAIS

Implantação de APIs: Container e Cloud

O primeiro passo para levar uma API de Machine Learning do ambiente de desenvolvimento para produção é a containerização da aplicação. O principal problema que queremos resolver é: funciona na minha máquina, mas não funciona no servidor!



Figura 1 – Docker
Fonte: Google Imagens (2026)

Containers são uma forma leve de empacotar a aplicação junto com todas as suas dependências e configurações, isolando-a de outras aplicações no sistema. Diferente de uma máquina virtual completa, um contêiner compartilha o kernel do sistema host, consumindo bem menos recursos e permitindo uma execução mais eficiente. O Docker, por exemplo, empacota sua API Python + as bibliotecas utilizadas em uma “caixa” que roda em qualquer computador ou servidor.

CARACTERÍSTICA	MÁQUINA VIRTUAL	CONTAINER
Tamanho	GBs	MBs
Início	Minutos	Segundos
Isolamento	Sistema completo	Apenas app
Portabilidade	Limitada	Alta

Tabela 1 – Características, máquinas virtuais e containers
Fonte: Elaborado pelo autor (2026)

Para nossa API Iris, o Docker garante que ela vai funcionar no Render, AWS, Azure ou qualquer outro lugar, exatamente como funciona na sua máquina.

Dockerfile

Para containerização da nossa API FastAPI de ML, criamos um arquivo Dockerfile descrevendo como montar a imagem. Nele definimos, por exemplo: a imagem base do Python, o diretório de trabalho, instalamos as dependências do requirements.txt e copiamos o código da aplicação para dentro do contêiner. Por fim, especificamos o comando de inicialização da API quando o contêiner for executado.

Em cenários de produção com maior carga, é comum integrar o Gunicorn como gerenciador de processos para rodar múltiplos workers Uvicorn em paralelo (aproveitando vários núcleos de CPU). Assim, o Dockerfile poderia usar um comando como `gunicorn -k uvicorn.workers.UvicornWorker -w 4 -b 0.0.0.0:80 app.main:app`, onde o Gunicorn inicia 4 processos Uvicorn, combinando alta concorrência assíncrona com monitoramento de processos.

Hands On - Preparando a API Iris para Produção

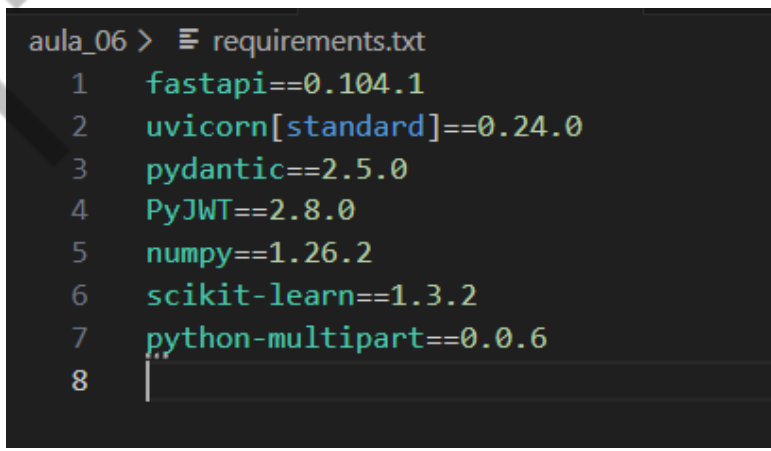
O primeiro passo vai ser organizar o nosso projeto, afinal não queremos que a aplicação seja um monolito de difícil manutenção.

Estrutura de pastas do projeto:

```
api-iris-producao/
├── app/
│   ├── __init__.py
│   ├── main.py          # Arquivo principal da API FastAPI
│   ├── auth.py          # Módulo com funções para JWT (JSON Web Token)
│   └── models/
│       ├── modelo_iris.pkl # Arquivo do modelo de machine learning (Iris)
│       └── classes_iris.pkl # Arquivo com as classes do modelo (Iris)
├── requirements.txt      # Lista de dependências do projeto
├── Dockerfile            # Instruções para a criação da imagem Docker
├── .env.example          # Exemplo de arquivo de variáveis de ambiente
└── README.md             # Documentação principal do projeto
```

Arquivo requirements.txt

Aqui vamos listar todas as dependências do projeto com versões fixas para podermos reproduzir com segurança.



```
aula_06 > requirements.txt
1 fastapi==0.104.1
2 uvicorn[standard]==0.24.0
3 pydantic==2.5.0
4 PyJWT==2.8.0
5 numpy==1.26.2
6 scikit-learn==1.3.2
7 python-multipart==0.0.6
8
```

Figura 2 - Requirements.txt
Fonte: Elaborado pelo autor (2026)

Se você ainda não tiver o Docker Desktop instalado, faça download no site oficial para seu sistema operacional. O Docker é gratuito para estudantes e pequenas empresas e vai ser necessário para montarmos o Dockerfile.

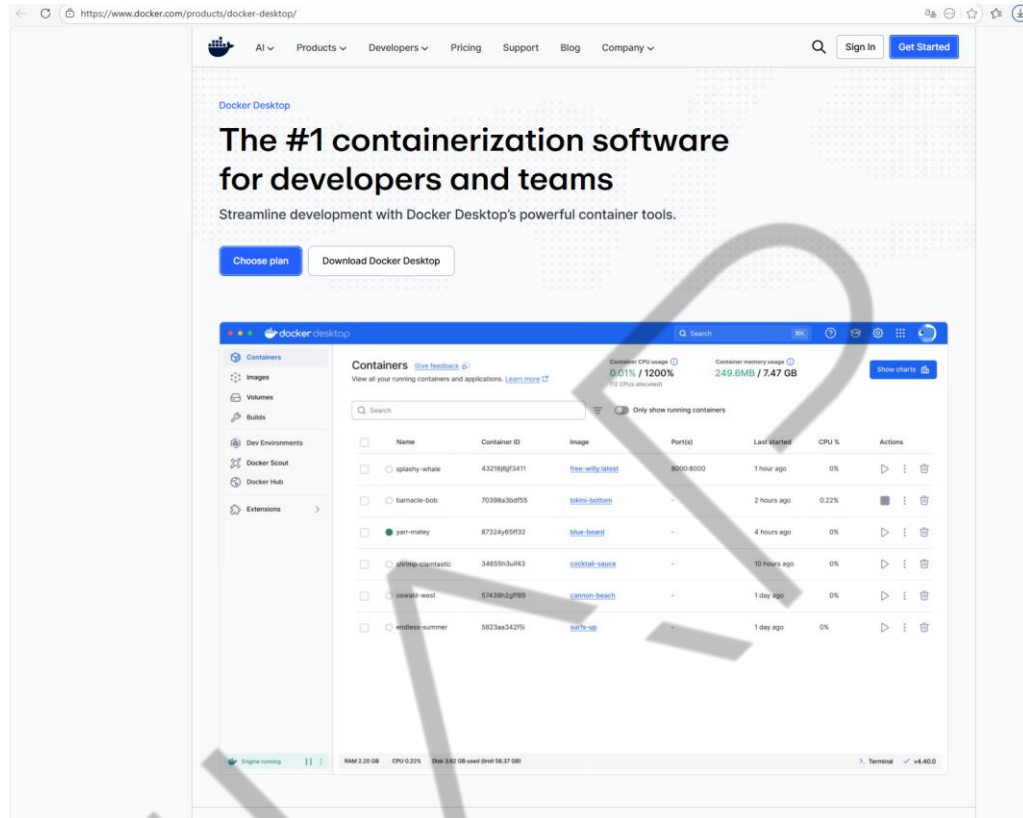


Figura 3 - Docker – Desktop
Fonte: Elaborado pelo autor (2026)

Vamos seguir a modularização com nosso arquivo auth.py, que agora vai conter as funções para criar e validar os tokens jwt:

```
"""
Modulo de Autenticacao JWT
Funcoes para criar e validar tokens JWT
"""

import os
from datetime import datetime, timedelta

import jwt
from fastapi import Depends, HTTPException
from fastapi.security import HTTPBearer,
HTTPAuthorizationCredentials
```



```
#
=====
=====
# CONFIGURACOES
#
=====
=====
SECRET_KEY = os.getenv("JWT_SECRET", "dev-secret-change-
in-production")
ALGORITHM = "HS256"
TOKEN_EXPIRE_MINUTES =
int(os.getenv("TOKEN_EXPIRE_MINUTES", "30"))

# Usuarios (em producao, use banco de dados!)
USERS_DB = {
    "admin": {"password": os.getenv("ADMIN_PASSWORD",
"admin123"), "role": "admin"},
    "user": {"password": os.getenv("USER_PASSWORD",
"user123"), "role": "user"},
}

#
=====
=====
# SEGURANCA
#
=====
=====
security = HTTPBearer()

def create_token(username: str, role: str) -> str:
    """Cria um token JWT com expiracao."""
    expire = datetime.utcnow() +
timedelta(minutes=TOKEN_EXPIRE_MINUTES)
    payload = {"sub": username, "role": role, "exp":
expire}
    return jwt.encode(payload, SECRET_KEY,
algorithm=ALGORITHM)

def get_current_user(credentials:
HTTPAuthorizationCredentials = Depends(security)) -> dict:
    """Valida o token JWT e retorna o usuario."""
    token = credentials.credentials
    try:
        payload = jwt.decode(token, SECRET_KEY,
algorithms=[ALGORITHM])
        return {"username": payload["sub"], "role":
payload["role"]}
    except:
```

```

        except jwt.ExpiredSignatureError:
            raise HTTPException(status_code=401, detail="Token
expirado")
        except jwt.InvalidTokenError:
            raise HTTPException(status_code=401, detail="Token
invalido")

```

```

def authenticate_user(username: str, password: str) -> dict
| None:
    """Verifica credenciais do usuario."""
    user = USERS_DB.get(username)
    if user and user["password"] == password:
        return {"username": username, "role": user["role"]}
    return None

```

Código-fonte 1 – Arquivo auth.py com funções para criar e validar os tokens jwt
 Fonte: Elaborado pelo autor (2026)

No arquivo main.py vamos manter todas as rotas e configurações. Posteriormente também iremos modularizar ele, mas para simplificar o exemplo do docker manteremos elas no mesmo arquivo:

```

"""
API Iris com JWT - Versao Producao (Modularizada)
Pronta para deploy em container/cloud
"""
import os
import pickle
import logging
from pathlib import Path

import numpy as np
from fastapi import Depends, FastAPI, HTTPException
from pydantic import BaseModel, Field

# Importa funcoes do modulo auth
from app.auth import (
    create_token,
    get_current_user,
    authenticate_user,
    TOKEN_EXPIRE_MINUTES,
)

#
=====
=====

```

```

# CONFIGURACOES
#
=====
=====
ENVIRONMENT = os.getenv("ENVIRONMENT", "development")

#
=====
=====
# LOGGING
#
=====
=====
logging.basicConfig(
    level=logging.INFO,
    format="%(asctime)s | %(levelname)s | %(message)s"
)
logger = logging.getLogger(__name__)

#
=====
=====
# SCHEMAS
#
=====
=====
class LoginRequest(BaseModel):
    username: str
    password: str

class TokenResponse(BaseModel):
    access_token: str
    token_type: str = "bearer"
    expires_in: int

class IrisRequest(BaseModel):
    sepal_length: float = Field(..., ge=0, le=10,
description="Comprimento da sepala (cm)")
    sepal_width: float = Field(..., ge=0, le=10,
description="Largura da sepala (cm)")
    petal_length: float = Field(..., ge=0, le=10,
description="Comprimento da petala (cm)")
    petal_width: float = Field(..., ge=0, le=10,
description="Largura da petala (cm)")

class IrisResponse(BaseModel):

```

```

        sucesso: bool
        classe: str
        probabilidades: dict
        usuario: str

#
=====
# CARREGAMENTO DO MODELO
#
=====
MODEL_PATHS = [
    Path("app/models/modelo_iris.pkl"),      # Estrutura
modularizada
    Path("models/modelo_iris.pkl"),          # Alternativa
    Path("/app/app/models/modelo_iris.pkl"), # Dentro do
container
    Path("modelo_iris.pkl"),                  # Raiz
(fallback)
]

modelo = None
classes = None

for model_path in MODEL_PATHS:
    if model_path.exists():
        with open(model_path, 'rb') as f:
            modelo = pickle.load(f)
            classes_path = model_path.parent /
"classes_iris.pkl"
            if classes_path.exists():
                with open(classes_path, 'rb') as f:
                    classes = pickle.load(f)
            logger.info(f"Modelo carregado de: {model_path}")
            break

MODELO_OK = modelo is not None and classes is not None
if not MODELO_OK:
    logger.warning("Modelo NAO encontrado! API funcionara
sem predicacao.")

#
=====
# APLICACAO FASTAPI
#
=====
=====

```

```

app = FastAPI(
    title="API Iris com JWT",
    description="API de classificacao de flores Iris com
autenticacao JWT. Deploy em producao.",
    version="2.0.0",
    docs_url="/docs",
    redoc_url="/redoc",
)

@app.get("/")
def home():
    """Informacoes da API."""
    return {
        "api": "Iris Classifier",
        "versao": "2.0.0",
        "ambiente": ENVIRONMENT,
        "modelo_carregado": MODELO_OK,
        "docs": "/docs",
    }

@app.get("/health")
def health():
    """Health check para monitoramento."""
    return {
        "status": "healthy" if MODELO_OK else "degraded",
        "modelo": MODELO_OK,
        "ambiente": ENVIRONMENT,
    }

@app.post("/login", response_model=TokenResponse)
def login(credentials: LoginRequest):
    """Faz login e retorna token JWT."""
    user = authenticate_user(credentials.username,
credentials.password)

    if not user:
        logger.warning(f"Login falhou para:
{credentials.username}")
        raise HTTPException(status_code=401,
detail="Usuario ou senha incorretos")

    token = create_token(user["username"], user["role"])
    logger.info(f"Login bem-sucedido: {user['username']}")
    return TokenResponse(access_token=token,
expires_in=TOKEN_EXPIRE_MINUTES * 60)

@app.get("/me")

```

```
def get_me(current_user: dict = Depends(get_current_user)):
    """Retorna informacoes do usuario logado."""
    return current_user

@app.post("/predict", response_model=IrisResponse)
def predict(payload: IrisRequest, current_user: dict =
Depends(get_current_user)):
    """Faz predicao (requer autenticacao)."""
    if not MODELO_OK:
        raise HTTPException(status_code=503,
detail="Modelo nao disponivel")

    features = np.array([[
        payload.sepal_length, payload.sepal_width,
        payload.petal_length, payload.petal_width
    ]])

    pred_idx = modelo.predict(features)[0]
    probs = modelo.predict_proba(features)[0]
    classe = classes[pred_idx]

    logger.info(f"Predicao: {classe} por
{current_user['username']}")

    return IrisResponse(
        sucesso=True,
        classe=classe,
        probabilidades={classes[i]: round(float(p), 4) for
i, p in enumerate(probs)},
        usuario=current_user["username"]
    )
```

Código-fonte 2 – Arquivo main.py
 Fonte: Elaborado pelo autor (2026)

Criando o Dockerfile

Agora vamos criar o nosso Dockerfile; essa é a “receita” para construir o container:

```
FROM          # Imagem base (Python)
WORKDIR       # Diretorio de trabalho
COPY          # Copiar arquivos
RUN          # Executar comandos (pip install)
EXPOSE        # Porta que a API usa
CMD           # Comando para iniciar a API
```

Código-fonte 3 – Dockerfile (DSL do Docker)
 Fonte: Elaborado pelo autor (2026)

```

aula_06 > Dockerfile > ...
1  # =====
2  # Dockerfile para API Iris com JWT
3  # =====
4
5  # Imagem base: Python 3.11 slim (menor e mais segura)
6  FROM python:3.11-slim
7
8  # Definir diretório de trabalho dentro do container
9  WORKDIR /app
10
11 # Copiar arquivo de dependências primeiro (aproveita cache do Docker)
12 COPY requirements.txt .
13
14 # Instalar dependências
15 RUN pip install --no-cache-dir -r requirements.txt
16
17 # Copiar o pacote app inteiro (código + modelos)
18 COPY app/ ./app/
19
20 # Expor a porta (documentação, Render usa $PORT)
21 EXPOSE 8000
22
23 # Comando para iniciar a API (agora app.main:app)
24 # Usamos $PORT para compatibilidade com Render/Heroku
25 CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
26
27
    
```

Figura 4 – Dockerfile
 Fonte: Elaborado pelo autor (2026)

Agora que temos nosso Docker configurado, vamos testar localmente antes de fazer o deploy no Render.

```

```bash
1. Construir a imagem
docker build -t api-iris-jwt .

2. Rodar o container
docker run -d -p 8000:8000 --name iris-container api-iris-
jwt

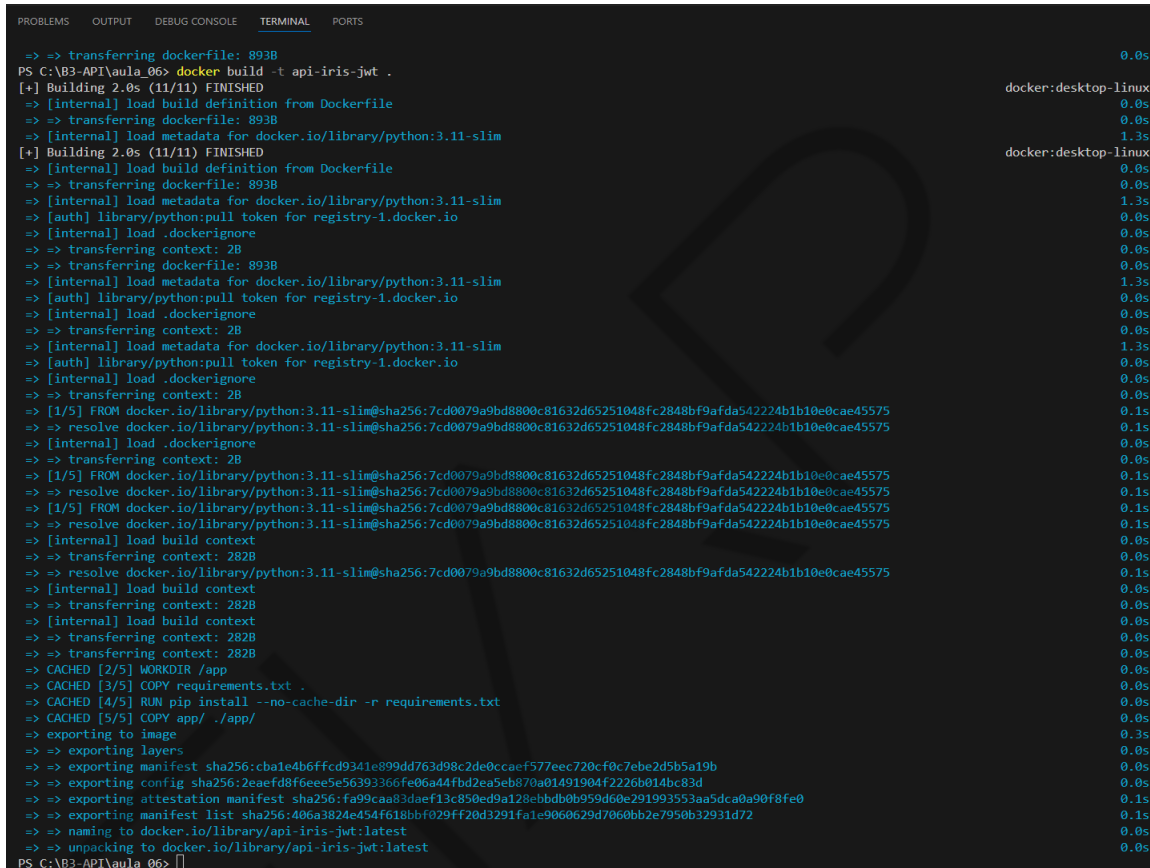
3. Testar
curl http://localhost:8000/
curl http://localhost:8000/health

4. Ver logs
docker logs iris-container

```

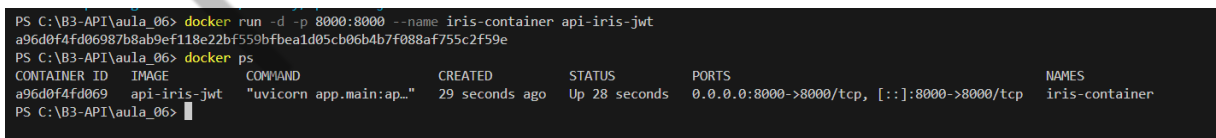
```
5. Parar e remover
docker stop iris-container
docker rm iris-container
\\ \\
```

Código-fonte 4 – Teste local (bash)  
Fonte: Elaborado pelo autor (2026)



```
=> => transferring dockerfile: 893B
PS C:\B3-API\aula_06> docker build -t api-iris-jwt .
[+] Building 2.0s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 893B
=> [internal] load metadata for docker.io/library/python:3.11-slim
[+] Building 2.0s (11/11) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 893B
=> [internal] load metadata for docker.io/library/python:3.11-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> => transferring dockerfile: 893B
=> [internal] load metadata for docker.io/library/python:3.11-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load metadata for docker.io/library/python:3.11-slim
=> [auth] library/python:pull token for registry-1.docker.io
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:7cd0079a9bd8800c81632d65251048fc2848bf9afda542224b1b10e0cae45575
=> => resolve docker.io/library/python:3.11-slim@sha256:7cd0079a9bd8800c81632d65251048fc2848bf9afda542224b1b10e0cae45575
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:7cd0079a9bd8800c81632d65251048fc2848bf9afda542224b1b10e0cae45575
=> => resolve docker.io/library/python:3.11-slim@sha256:7cd0079a9bd8800c81632d65251048fc2848bf9afda542224b1b10e0cae45575
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:7cd0079a9bd8800c81632d65251048fc2848bf9afda542224b1b10e0cae45575
=> => resolve docker.io/library/python:3.11-slim@sha256:7cd0079a9bd8800c81632d65251048fc2848bf9afda542224b1b10e0cae45575
=> [internal] load build context
=> => transferring context: 282B
=> [internal] load build context
=> => transferring context: 282B
=> [internal] load build context
=> => transferring context: 282B
=> [internal] load build context
=> => transferring context: 282B
=> CACHED [2/5] WORKDIR /app
=> CACHED [3/5] COPY requirements.txt .
=> CACHED [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> CACHED [5/5] COPY app/ ./app/
=> exporting to image
=> => exporting layers
=> => exporting manifest sha256:cbale4b6ffcd9341e899dd763d98c2de0ccaef577ec720cf0c7e2d5b5a19b
=> => exporting config sha256:2aeafd8f6ee5e56393366fe06a44fbd2ea5eb870a01491904f2226b014bc83d
=> => exporting attestation manifest sha256:fa99caa83daef13c850ed9a128ebdb0b959d60e29199353aa5dca0a90f8fe0
=> => exporting manifest list sha256:406a3824e454f618bbf029ff20d3291fa1e9060629d7060bb2e7950b32931d72
=> => naming to docker.io/library/api-iris-jwt:latest
=> => unpacking to docker.io/library/api-iris-jwt:latest
PS C:\B3-API\aula_06>
```

Figura 5 - Docker build  
Fonte: Elaborado pelo autor (2026)



```
PS C:\B3-API\aula_06> docker run -d -p 8000:8000 --name iris-container api-iris-jwt
a96d0f4fd06987b8ab9ef118e22bf559bfbeald05cb06b4b7f088af755c2f59e
PS C:\B3-API\aula_06> docker ps
CONTAINER ID IMAGE COMMAND CREATED STATUS PORTS NAMES
a96d0f4fd069 api-iris-jwt "uvicorn app.main:ap..." 29 seconds ago Up 28 seconds 0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp iris-container
PS C:\B3-API\aula_06>
```

Figura 6 - Docker RUN  
Fonte: Elaborado pelo autor (2026)

Podemos verificar no terminal o container rodando com o comando docker ps, mas também é possível monitorar pelo aplicativo docker desktop:



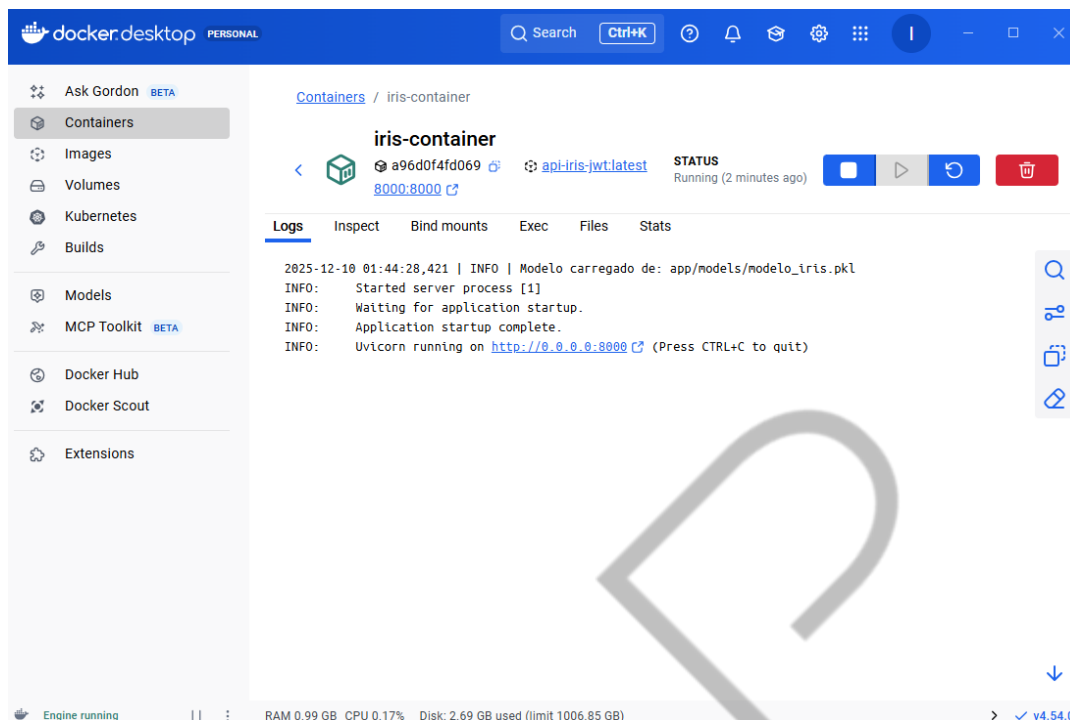


Figura 7 - Docker desktop  
Fonte: Elaborado pelo autor (2026)

## Opções de Deploy em Nuvem

Com nossa API containerizada e validada localmente, o próximo passo é implantá-la em ambiente cloud real. Há diversas estratégias para deploy na nuvem, desde infraestrutura-como-Serviço (IaaS) até Plataformas-como-Serviço (PaaS) e soluções serverless. É importante entender as diferenças para escolher a mais adequada.

A implantação de APIs pode ser feita em diferentes modelos de infraestrutura, cada um com suas vantagens e desvantagens:

### Máquinas Virtuais (IaaS):

- **Descrição:** consiste em provisionar e gerenciar uma Máquina Virtual (VM) em um provedor de nuvem (como AWS EC2, Azure VM, Google Cloud VM) para rodar a API.
- **Controle e Flexibilidade:** oferece o máximo controle sobre o ambiente (sistema operacional, hardware virtual), permitindo rodar o contêiner Docker da API diretamente na VM (após instalar o Docker Engine) ou fazer o deploy manual do código.

- **Desvantagens:** exige maior esforço de configuração e manutenção, incluindo gestão de atualizações do SO, escalabilidade manual (usando mais VMs atrás de um load balancer) e configurações de rede/segurança. Além disso, gera custos fixos, pois o servidor precisa estar continuamente ativo, mesmo quando a aplicação está ociosa.

### Plataformas Serverless (FaaS):

- **Descrição:** utiliza computação sob demanda, como AWS Lambda com API Gateway. O código da API (função) é executado apenas em resposta às requisições, sem a necessidade de um servidor fixo rodando continuamente.
- **Vantagens:** auto-escalabilidade e modelo de pagamento por uso (apenas pelo tempo de execução), eliminando custos de inatividade.
- **Integração com FastAPI:** embora o FastAPI exija servidores ASGI persistentes, pode ser adaptado para rodar em ambientes Lambda usando handlers como a biblioteca Mangum, que traduz eventos do API Gateway para chamadas ASGI.
- **Desafios:** limites de tempo de execução (Lambda tipicamente até 15 minutos), o fenômeno cold start (atraso na inicialização após um período de inatividade) e a necessidade de otimizar a aplicação para inicialização rápida. É atraente para aplicações com uso intermitente ou escalabilidade imprevisível.

### Plataformas como Serviço (PaaS):

- **Descrição:** serviços gerenciados (ex.: Heroku, Railway, Render, Azure App Services) que fornecem um meio-termo, abstraindo grande parte da gestão de infraestrutura.
- **Facilidade de Deploy:** o usuário entrega o código-fonte ou uma imagem Docker e a plataforma se encarrega de provisionar e gerenciar os contêineres (dynos) e a infraestrutura subjacente (servidores web, SO, certificados HTTPS). Por exemplo, o Heroku simplifica o deploy ao aceitar a imagem Docker ou construir a aplicação a partir de um Dockerfile.

- **Trade-off:** menor controle sobre o ambiente em comparação com IaaS. Embora ofereça escalonamento automatizado opcional, pode ter custos mais altos para cargas de trabalho constantes, pois cobra por instâncias ativas continuamente.

## Containerização como Facilitador

A containerização padroniza o ambiente da API, facilitando a portabilidade entre essas opções de deploy. Uma mesma imagem Docker pode ser usada em testes locais, em uma VM, em um cluster Kubernetes ou enviada para um PaaS que suporte containers. Essa uniformidade entre desenvolvimento e produção minimiza erros de configuração. A escolha final dependerá dos requisitos específicos de escala e do nível de gerenciamento de infraestrutura desejado.

## Deploy no Render (Minha preferência Gratuita)

O Render é uma plataforma PaaS (Platform as a Service) moderna que oferece recursos cruciais para a implantação de APIs:

- **Gratuidade:** possui um plano gratuito, ideal para projetos de pequeno porte.
- **Integração com Git:** realiza deploy automático, diretamente via GitHub.
- **Suporte a Container:** oferece suporte nativo ao Docker.
- **Segurança:** inclui SSL gratuito (HTTPS) para conexões seguras.
- **Monitoramento:** apresenta logs em tempo real.

## Opções de Implantação (Deploy) no Render

MÉTODO	DESCRIÇÃO	CENÁRIO DE USO IDEAL
Docker	O Render utiliza o Dockerfile presente no repositório para executar o comando docker build.	Quando é necessário ter controle total sobre o ambiente de runtime.
Nativo (Native)	O Render detecta a linguagem (ex.: Python) e instala as dependências automaticamente.	Para a opção mais simples, sem a necessidade de um Dockerfile.
Blueprint	Utiliza um arquivo render.yaml importado via dashboard para configurar os serviços.	Implementação de Infraestrutura como Código (IaC).

Tabela 2 – Opções de Implantação (Deploy) no Render  
Fonte: Elaborado pelo autor (2026)

### Método de Implantação Escolhido

Neste projeto, será utilizado o Docker. Como o Dockerfile já foi criado, o processo no Render será o seguinte:

- Detecção do Dockerfile no repositório.
- Execução do docker build (o mesmo comando utilizado localmente).
- Execução do container na nuvem.

**Importante:** o arquivo render.yaml é opcional e serve para a configuração via método "Blueprint". Se o Web Service for criado manualmente e o Dockerfile estiver presente, o Render automaticamente adota o método de implantação via Docker.

O arquivo render.yaml acaba não sendo utilizado no nosso caso, mas ele está presente em nosso projeto como referência:

```

aula_06 > ! render.yaml
1 # =====
2 # Render Blueprint - Configuracao de Infraestrutura
3 # =====
4 # Este arquivo define como o Render deve fazer deploy da sua aplicacao
5 # Docs: https://render.com/docs/blueprint-spec
6
7 services:
8 - type: web
9 name: api-iris-jwt
10 env: python
11 region: oregon
12 plan: free
13
14 # Comando para iniciar a API (estrutura modularizada)
15 startCommand: uvicorn app.main:app --host 0.0.0.0 --port $PORT
16
17 # Variaveis de ambiente
18 envVars:
19 - key: ENVIRONMENT
20 value: production
21 - key: JWT_SECRET
22 generateValue: true
23 - key: TOKEN_EXPIRE_MINUTES
24 value: "60"
25 - key: PYTHON_VERSION
26 value: "3.11.0"
27
28 # Health check
29 healthCheckPath: /health
30

```

Figura 8 - Arquivo render.yaml  
Fonte: Elaborado pelo autor (2026)

- Caso ainda não tenha uma conta no Render, é só seguir esses passos: Acesse o site oficial: <https://render.com>
- Clique no botão "Get Started for Free".
- Recomendamos fazer login utilizando sua conta GitHub para simplificar o processo.

Criar o Serviço Web (Web Service)

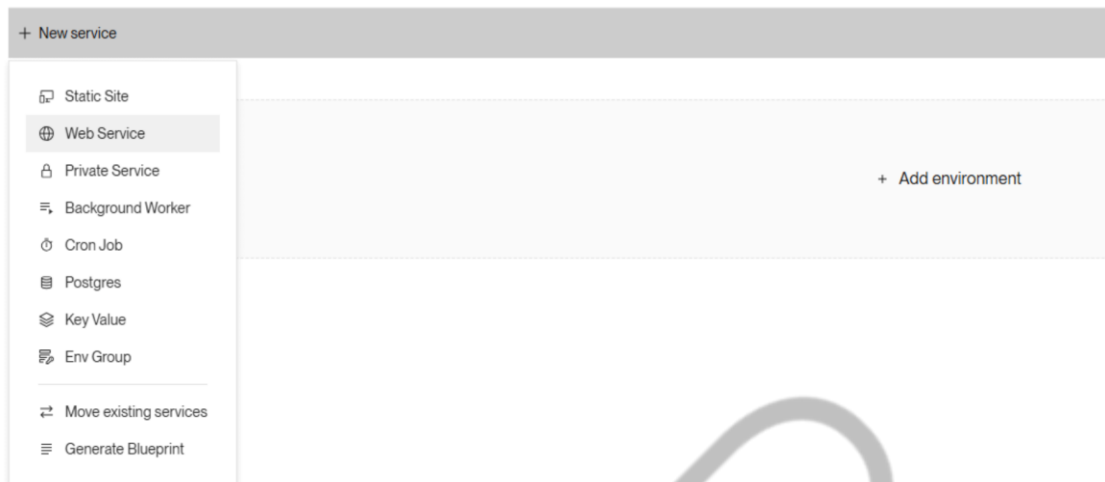


Figura 9 - Serviço Web (Web Service)  
Fonte: Elaborado pelo autor (2026)

- No dashboard do Render, clique em "New +" e selecione a opção "Web Service".
- Conecte a plataforma ao seu repositório no GitHub.
- Escolha o repositório que contém o código da sua API.
- Defina as configurações do serviço:
  - Nome: api-iris-jwt;
  - Região: Oregon (US West);
  - Branch: main;
  - Diretório Raiz (Root Directory): Defina como aula\_06 apenas se os arquivos de deploy estiverem localizados nessa subpasta;
  - Runtime: O Render detectará automaticamente o Docker;
  - Finalize clicando em "Create Web Service".

### New Web Service

**Source Code**

ishpedro / API-INFERENCIA-MODELOS [Edit](#)

**Name**

A unique name for your web service.

API-INFERENCIA-MODELOS

**Project** Optional

Add this web service to a [project](#) once it's created.

Contai-Reporter / Development

**Language**

Choose the [runtime environment](#) for this service.

Docker

**Branch**

The Git branch to build and deploy.

main

**Region**

Your services in the same [region](#) can communicate over a [private network](#). You currently have services running in [Oregon](#).

Oregon (US West) 1 existing service

[Deploy in a new region](#) +

**Root Directory** Optional

If set, Render runs commands from this directory instead of the repository root. Additionally, code changes outside of this directory do not trigger an auto-deploy. Most commonly used with a [monorepo](#).

./src

**Dockerfile Path**

The path to your service's Dockerfile, relative to the repo root. Defaults to `./Dockerfile`.

./

**Instance Type**

For hobby projects

Free	512 MB (RAM)	<a href="#">Upgrade to enable more features</a> Free instances spin down after periods of inactivity. They do not support SSH access, scaling, one-off jobs, or persistent disks. Select any paid instance type to enable these features.
\$0 / month	0.1 CPU	

Figura 10 – New Web Service  
Fonte: Elaborado pelo autor (2026)

Atenção: se o arquivo Dockerfile não estiver na raiz do repositório, mas sim em uma subpasta (ex: aula\_06/), é obrigatório configurar o Root Directory nas Settings do serviço para o caminho correto.

```

All logs v Search
Dec 9 07:44:47 PM -> It looks like we don't have access to your repo, but we'll try to clone it anyway.
Dec 9 07:44:47 PM -> Cloning from https://github.com/ishpedro/API-INFERENCIA-MODELOS
Dec 9 07:44:48 PM -> Checking out commit 80233c18701b04b75d7213c548e80f13830e9 in branch main
Dec 9 07:44:50 PM -> #1 [internal] load build definition from Dockerfile
Dec 9 07:44:50 PM -> #1 transferring dockerfile: 28 done
Dec 9 07:44:50 PM -> #1 DONE 0.0s
Dec 9 07:44:50 PM -> error: failed to solve: failed to read dockerfile: open Dockerfile: no such file or directory

```

Figura 11 - Root Directory  
Fonte: Elaborado pelo autor (2026)

**Build & Deploy**

**Repository**  
The repository used for your Web Service.  
https://github.com/lohpdro/API-INFERENCIA-MODELOS [Edit](#)

**Branch**  
The Git branch to build and deploy.  
main [Edit](#)

**Git Credentials**  
User providing the credentials to pull the repository.  
ioannis.eleftheriou.design@gmail.com (you) [Use My Credentials](#)

**Root Directory** Optional  
If set, Render runs commands from this directory instead of the repository root. Additionally, code changes outside of this directory do not trigger an auto-deploy. Most commonly used with a [monorepo](#).  
/aula\_06 [Cancel](#) [Save Changes](#)

**Build Filters**  
Include or ignore specific paths in your repo when determining whether to trigger an auto-deploy. Paths are relative to your repo's root directory. [Learn more](#) [Edit](#)

**Included Paths**  
Changes that match these paths will trigger a new build.  
[+ Add Included Path](#)

Figura 12 – Build & Deploy  
Fonte: Elaborado pelo autor (2026)

Não se esqueça de configurar as variáveis de ambiente:

**Environment Variables**  
Set environment-specific config and secrets (such as API keys), then read those values from your code. [Learn more](#)

Name	Value	Visibility
JWT_SECRET	*****	Secret
ENVIRONMENT	*****	Secret
TOKEN_EXPIRE_MINUTES	*****	Secret

[+ Add Environment Variable](#) [Add from env](#)

Figura 13 – Variáveis de Ambiente  
Fonte: Elaborado pelo autor (2026)

Depois de configurado, o render vai executar o docker build com nosso Dockerfile. Podemos acompanhar os logs em tempo real, agora basta aguardar o Deploy.



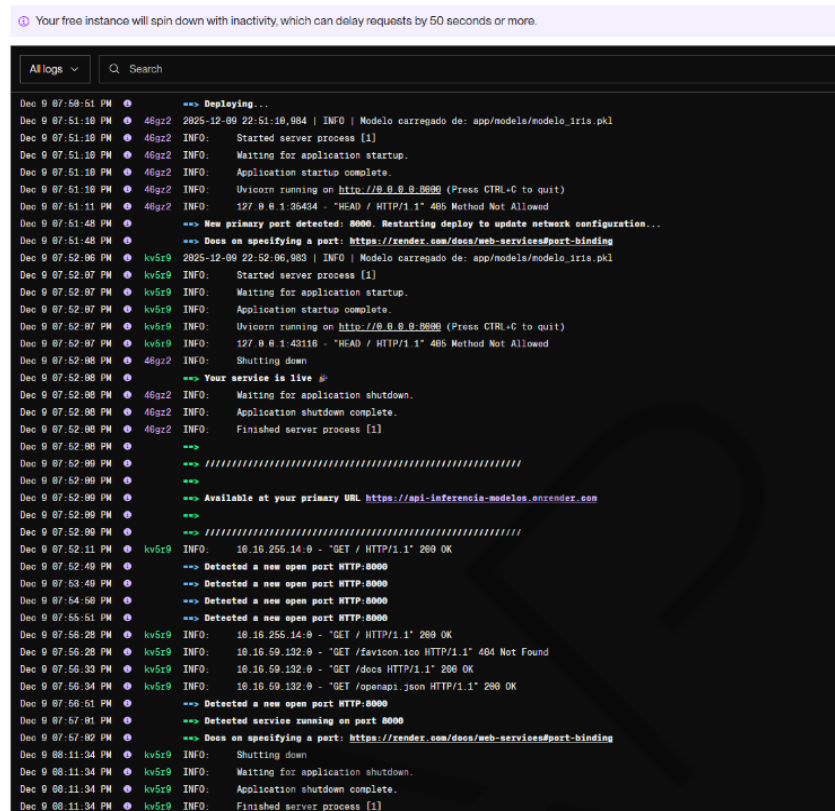


Figura 14 – Deploy  
Fonte: Elaborado pelo autor (2026)

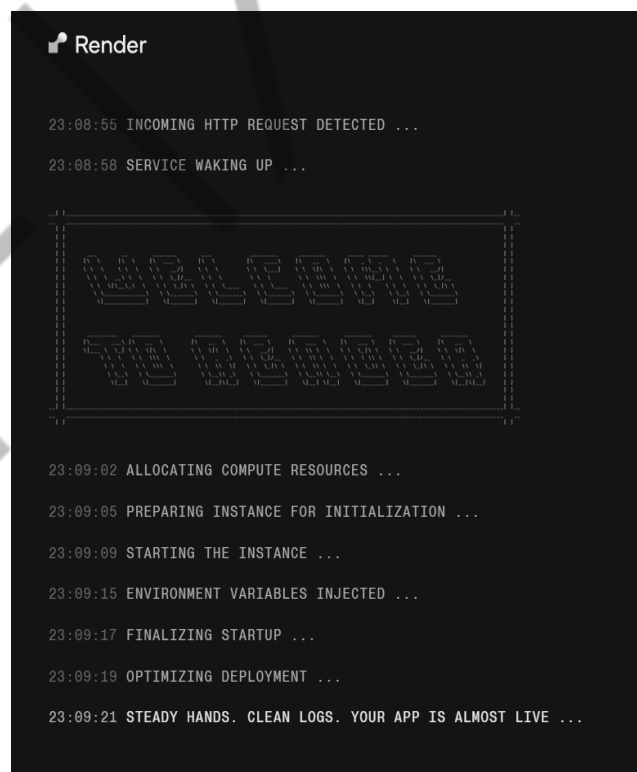


Figura 15 – Render  
Fonte: Elaborado pelo autor (2026)

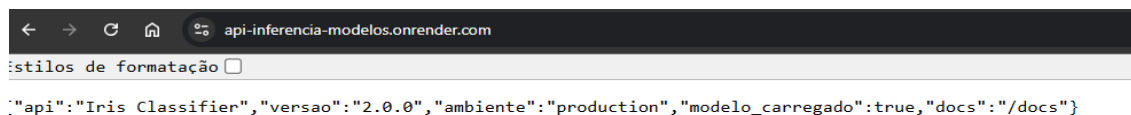


Figura 16 – Estilos de Formatação  
Fonte: Elaborado pelo autor (2026)

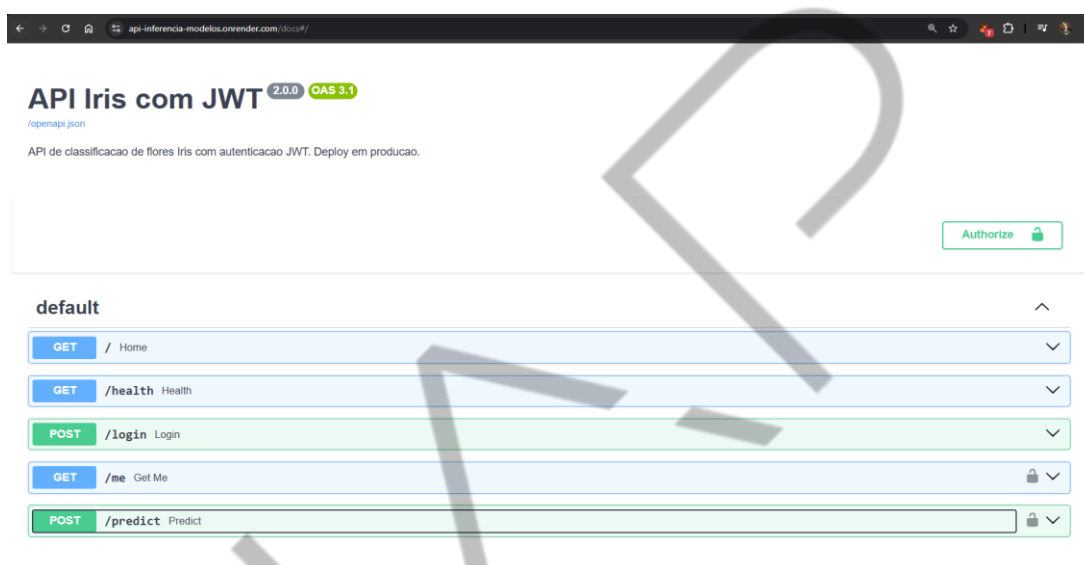


Figura 17 – API Iris com JWT  
Fonte: Elaborado pelo autor (2026)

Parabéns! Agora nossa API está implantada na nuvem e é acessível ao público em uma URL pública gratuita.

## Implementação de APIs: CI/CD e Implantação Contínua Automatizada

Ao mover uma API para um ambiente de produção, é altamente recomendável adotar práticas de CI/CD (Integração Contínua/Entrega Contínua). O objetivo é otimizar as atualizações e manter a qualidade do serviço.

Essencialmente, o CI/CD estabelece um pipeline automatizado: a cada nova alteração de código (por exemplo, um push para o branch principal), o sistema integra as modificações, executa testes rigorosos e, se aprovados, realiza a implantação (deploy) automática na nuvem. Este método elimina a necessidade de deploys

manuals e propensos a erros, garantindo que o ambiente de produção esteja sempre com a versão mais recente e verificada do código.

### Ferramentas e Exemplo Prático (GitHub Actions e Heroku)

Ferramentas como GitHub Actions (ou alternativas como GitLab CI, Jenkins) são comumente utilizadas para configurar esse fluxo. O processo envolve a criação de um workflow em formato YAML que define os passos (jobs) de construção (build), teste e implantação.

Em um cenário com Docker, o pipeline pode incluir o push da imagem para um registry (Docker Hub, ECR etc.) antes da implantação.

#### Exemplo de Workflow YAML para Deploy no Heroku:

O trecho YAML a seguir ilustra a configuração do GitHub Actions para um deploy no Heroku:

```
jobs:
 build:
 runs-on: ubuntu-latest
 steps:
 - uses: actions/checkout@v2
 - name: Install Heroku CLI
 run: |
 curl https://cli-assets.heroku.com/install.sh | sh
 - uses: akhileshns/heroku-deploy@v3.14.15
 with:
 heroku_api_key: ${ secrets.HEROKU_API_KEY }
 heroku_app_name: "NOME-DO-APP"
 heroku_email: "SEU-EMAIL"
```

Neste workflow, o job é executado em um ambiente Ubuntu a cada push. O fluxo inclui:

- Checkout do código.
- Instalação da CLI do Heroku.
- Execução da Ação de Deploy (heroku-deploy), onde são fornecidos dados sensíveis como o API key do Heroku, o nome do aplicativo e o e-mail de login.

É crucial notar que dados sensíveis (como a chave de API) são armazenados e referenciados de forma segura nos Secrets do GitHub (`secrets.HEROKU_API_KEY`).

Com esse pipeline ativo, um simples push no GitHub desencadeia a CI (execução de testes) seguida da entrega contínua (o deploy), reduzindo drasticamente o tempo de lançamento de features e minimizando o risco de falhas humanas.

## Alternativas e Escopo Profissional

Embora plataformas como Heroku ofereçam integração nativa (auto-deploy), configurar um pipeline YAML próprio oferece maior nível de personalização.

Em ambientes profissionais, etapas adicionais são comuns, como a execução de linters, testes unitários e de integração, construção de imagens Docker e implementação de estratégias avançadas de entrega gradual (como blue-green deployments). Contudo, um fluxo básico de "teste -> build -> deploy" já proporciona ganhos significativos em agilidade e confiabilidade no ciclo de vida da API.

## Monitoramento e Manutenção Pós-Deploy

Com a API rodando na nuvem, fechamos o ciclo desenvolver -> implantar -> operar. Porém, o trabalho não termina no deploy: é crucial monitorar a aplicação em produção e estar preparado(a) para manter e atualizar. Os pilares principais são:

- **Logs Acessíveis e Estruturados:**
  - Habilitar e acompanhar logs (via CLI ou add-ons/ferramentas cloud).

- Foco em erros e exceções não tratadas para análise de causa raiz (RCA).
- Centralização com soluções de observabilidade é a prática ideal.
- **Métricas de Desempenho:**
  - Monitorar CPU, memória, tempo de resposta (latência), throughput (RPS) e taxa de erros.
  - Utilizar dashboards da plataforma (Heroku Metrics, CloudWatch) ou APMs (NewRelic, Datadog).
  - Configurar alertas proativos para desvios de performance.
- **Disponibilidade e Tratamento de Erros:**
  - Monitorar o uptime (serviços como Pingdom) para garantir que o endpoint responda 200 OK.
  - Integrar ferramentas de captura de exceções (Sentry, Rollbar) para erros no backend.
  - Ter visibilidade de falhas (códigos de erro da plataforma, ex.: H10, R14 no Heroku) para rápida correção.
- **Escalabilidade e Otimização Contínua:**
  - Estar atento(a) à carga real de uso para escalar horizontalmente (mais instâncias/dynos).
  - Monitorar uso de recursos (memória, conexões de banco) para evitar falhas (OOM kill).
  - Ajustar configurações (workers, cache) com base nas métricas observadas (afinamento contínuo).
- **Manutenção e Segurança:**
  - Manter rotina de aplicação de patches de segurança e atualização de dependências (com testes prévios).
  - Planejar mecanismos de backup e documentar processos de rollback e monitoração.

Na próxima aula, vamos nos aprofundar em exemplos práticos de como configurar alertas, interpretar métricas de latência e taxa de erros e explorar as ferramentas de observabilidade mais comuns para garantir a estabilidade e a qualidade de serviço da nossa API em produção.

EMANDA

## O QUE VOCÊ VIU NESTA AULA?

Nesta aula, você aprendeu as etapas cruciais para migrar uma API de Machine Learning do ambiente local para a produção em escala, focando em containerização e deploy em nuvem.

Primeiro, você compreendeu a importância da Containerização com Docker, que empacota a API e suas dependências, resolvendo o problema de "funciona na minha máquina" e garantindo portabilidade.

Você montou o Dockerfile e testou a imagem Docker localmente. Em seguida, vimos as Opções de Deploy em Nuvem: IaaS (Máquinas Virtuais): oferece controle máximo, mas exige alta gestão de infraestrutura; FaaS (Serverless): oferece pagamento por uso e auto-escalabilidade, sendo ideal para cargas intermitentes; PaaS (Plataformas como Serviço): abstrai a infraestrutura, sendo o método escolhido no exemplo prático com o Render devido à sua facilidade, suporte nativo a Docker e plano gratuito.

O ciclo de vida da API foi complementado pela introdução à CI/CD (Integração Contínua/Entrega Contínua), mostrando como automatizar testes e deploys para garantir que as atualizações sejam rápidas e confiáveis.

Por fim, você viu que o trabalho continua após a implantação, ressaltando a importância do Monitoramento e Manutenção Pós-Deploy. O objetivo desta aula foi fornecer o conhecimento prático para implantar sua API de ML em um ambiente real de produção de forma padronizada e robusta.

## REFERÊNCIAS

GÉRÓN, A. **Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow**. Beijing: O'Reilly Media, 2023.

GIFT, N.; DEZA, A. **Practical MLOps: Operationalizing Machine Learning Models**. Sebastopol: O'Reilly Media, 2021.

LUBANOVIC, B. **FastAPI: Modern Python Web Development**. Sebastopol: O'Reilly Media, 2023.

NEWMAN, S. **Building Microservices: Designing Fine-Grained Systems**. Sebastopol: O'Reilly Media, 2021.



## PALAVRAS-CHAVE

JWT. Token. API. WEB HTTP. REST. API RESTful. GraphQL. gRPC. JSON.  
MLOps.

EMSE

EMSE